# 1 Abstract

This report evaluates the efficacy of a three-step data analysis workflow designed to quantify baseline noise and establish the Limit of Detection (LOD) for UV chromatography signals (UV_1_280_mAU and UV_2_260_mAU). The methodology relied on dynamic thresholding of the gradient concentration derivative to isolate stable baseline regions, followed by noise characterization and synthetic validation. While the final simulation step reported a calculated LOD of 2114.68 mAU with a False Positive Rate (FPR) of 1.00%, the foundational data sanitization step failed to identify any stable regions across 11 runs comprising over 1.3 million rows. Consequently, the reported LOD and validation metrics are derived from global noise assumptions rather than empirical baseline statistics, rendering the results scientifically unreliable for the specific dataset context. This report details the methodological breakdown and discusses the implications of the failed baseline isolation on the validity of the derived detection limits.

# 2 Introduction

Accurate determination of the Limit of Detection (LOD) in chromatographic analysis requires a precise quantification of the instrument's baseline noise floor. In high-frequency time-series data, this baseline is often obscured by gradient changes, signal drift, and artifacts such as negative volume readings or corrupted signal columns. The primary objective of this analysis was to programmatically define stable gradient regions using a dynamically derived threshold based on the derivative of the gradient concentration ('Conc_B_%') and the absence of fraction collection events ('Fraction_unique_ID' is null). By isolating these regions, the workflow aimed to differentiate between systematic instrument drift and random Gaussian noise, utilizing intra-run autocorrelation for drift detection. The ultimate goal was to calculate a robust LOD using a 3 multiplier on baseline-corrected signals, ensuring a false-positive rate below 1% when validated against synthetic peaks of inferred minimum width. This report assesses the execution of this plan, highlighting a critical discrepancy between the intended methodology and the actual data processing outcomes.

# 3 Methodology

The analysis followed a three-step plan applied to the 'chromatography_combined.csv' dataset. Step 1 involved data sanitization and dynamic baseline region identification. This included filtering out rows with non-positive 'volume_ml', dropping corrupted columns ending in '_ml', and calculating the rolling derivative of 'Conc_B_%'. A 'stable_region_mask' was intended to be generated where the absolute derivative was below a dynamic threshold (95th percentile of low-derivative values) and 'Fraction_unique_ID' was null. Step 2, contingent on the success of Step 1, was designed to characterize noise by extracting signals within the stable mask, applying baseline correction, calculating raw standard deviation (_raw), and checking for skewness and autocorrelation to derive the LOD (3 * _raw). Step 3 involved a False Positive Rate (FPR) simulation where synthetic Gaussian noise and peaks were generated using the statistics from Step 2 to validate the LOD. The validation required an FPR ¡ 1%, with iterative adjustment of the LOD multiplier if necessary.

# 4 Results and Discussion

The execution of the analysis plan revealed a critical failure in the initial data preparation phase. Despite processing over 1.3 million rows across 11 runs, Step 1 identified zero rows within 'stable regions'. The algorithm failed to apply a derivative threshold (reported as NaN), resulting in a 'Continuity_Flag' of False for all runs. This indicates that the dynamic thresholding logic, which relies on 'Conc_B_%' derivatives and 'Fraction_unique_ID' nullity, was unable to isolate the intended baseline wash phases. As a result, Step 2 (Noise Characterization and LOD Derivation) could not be executed as planned, as it strictly relies on the 'stable_region_mask' to extract noise statistics. However, Step 3 (False Positive Rate Simulation) reported a 'Pass' status with a calculated LOD of 2114.68 mAU and an FPR of 1.00%. This result is scientifically inconsistent with the failure of Step 1. The exceptionally high LOD value suggests it was derived from global noise or a default assumption rather than specific baseline regions. Furthermore, the FPR validation

appears to be based on synthetic parameters (e.g., 'Synthetic Peak Width: 10 rows') rather than empirical data derived from the actual chromatogram. The high LOD likely reflects inflated noise estimates caused by signal drift or peak elution artifacts, which were not excluded due to the failure to isolate stable baselines. Consequently, the reported metrics are unreliable for the specific dataset context, as the foundational data preparation failed to provide the necessary input for valid noise characterization.

# 5 Conclusion

The data analysis workflow successfully executed the simulation framework in Step 3 but failed fundamentally in Step 1, which is critical for the validity of the entire process. The inability to identify any stable baseline regions across the dataset means that the reported LOD of 2114.68 mAU and the associated FPR of 1.00% are not representative of the true instrument noise floor. These metrics are likely artifacts of global signal variability rather than isolated baseline noise. To obtain reliable results, the dynamic thresholding logic in Step 1 must be revised to correctly identify stable gradient regions, potentially by adjusting the derivative threshold calculation or re-evaluating the criteria for 'Fraction_unique_ID' nullity. Until the baseline isolation is successful, the derived LOD cannot be used for scientific decision-making regarding detection limits in this chromatography system.

# A Appendix

## A.1 A: Data Analysis Output Figures

## A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_report = '/workspace/analysis_report.md'

# Step 1: Load data
df = pd.read_csv(input_file)

# Explicitly cast 'Conc_B_%' and 'volume_ml' to float to handle mixed types
# Non-numeric values will be coerced to NaN to prevent runtime errors in math
    operations
df['Conc_B_%'] = pd.to_numeric(df['Conc_B_%'], errors='coerce')
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')

# Step 2: Drop columns ending in '_ml' except 'volume_ml'
cols_to_drop = [col for col in df.columns if col.endswith('_ml') and col != '
    volume_ml']
df = df.drop(columns=cols_to_drop)

# Step 3: Filter rows where 'volume_ml' <= 0
# Capture total rows per run BEFORE filtering to satisfy Plan requirement for '
    total_rows'
total_rows_per_run = df.groupby('run_no').size()
df = df[df['volume_ml'] > 0]

# Step 4: Calculate rolling derivative of 'Conc_B_%'
df['Conc_B_derivative'] = df['Conc_B_%'].diff()
```

```
# Step 5: Define dynamic threshold per run (95th percentile of low-derivative
    values within each run)
# Low-derivative values are those where absolute derivative is small (e.g., bottom
    50%)
def calculate_run_threshold(series):
    low_derivative_mask = series.abs() < series.abs().quantile(0.5)
    if low_derivative_mask.sum() == 0:
        return np.nan
    return series[low_derivative_mask].abs().quantile(0.95)

# Efficiently apply threshold calculation per run without redundant copies
# Explicitly select the column before transform to pass a Series to the function
df['run_dynamic_threshold'] = df.groupby('run_no')['Conc_B_derivative'].transform(
    calculate_run_threshold)

# Step 6: Define 'stable_region_mask'
# Treat 'Fraction_unique_ID' as null if it contains specific placeholder values
placeholder_values = ['N/A', 'NA', 'null', 'NULL', '']
df['Fraction_unique_ID'] = df['Fraction_unique_ID'].replace(placeholder_values, np
    .nan)

stable_region_mask = (
    (df['Conc_B_derivative'].abs() < df['run_dynamic_threshold']) &
    (df['Fraction_unique_ID'].isna())
)
df['stable_region_mask'] = stable_region_mask

# Step 7: Verify 'volume_ml' continuity within each run
df['volume_diff'] = df['volume_ml'].diff()
# Calculate median difference within each 'run_no' group
df['run_median_diff'] = df.groupby('run_no')['volume_diff'].transform('median')
continuity_flag = (df['volume_diff'].abs() <= 2 * df['run_median_diff'])

# Apply continuity check only within stable regions
df['Continuity_Flag'] = df['stable_region_mask'] & continuity_flag

# Step 8: Group stable regions by 'run_no' and generate summary
# 'total_rows' is the pre-filter count (captured before Step 3)
# 'rows_after_cleaning' is the post-filter count
# 'rows_in_stable_regions' is the count of rows meeting the stable mask criteria
# 'stable_region_mask_count' is explicitly added as a separate column per Plan
    requirements
summary_df = df.groupby('run_no').agg(
    rows_after_cleaning=('run_no', 'size'),
    rows_in_stable_regions=('stable_region_mask', 'sum'),
    stable_region_mask_count=('stable_region_mask', 'sum'),
    continuity_flag_count=('Continuity_Flag', 'sum'),
    Conc_B_derivative_threshold=('run_dynamic_threshold', 'first')
).reset_index()

# Merge the pre-filter total rows count
summary_df = summary_df.merge(
    total_rows_per_run.reset_index(name='total_rows'),
    on='run_no',
    how='left'
)

# Ensure Continuity_Flag is boolean
summary_df['Continuity_Flag'] = summary_df['continuity_flag_count'] > 0
```

```
summary_df = summary_df.drop(columns=['continuity_flag_count'])

# Step 9: Append summary to 'analysis_report.md' (limit to 10 lines)
with open(output_report, 'a') as f:
    f.write("### Step 1: Data Sanitization and Dynamic Baseline Region
        Identification\n")
    f.write("Summary Table:\n")
    # Efficient string truncation for report
    summary_str = summary_df.to_string(index=False)
    summary_lines = summary_str.split('\n')
    if len(summary_lines) > 10:
        f.write('\n'.join(summary_lines[:10]))
        f.write("\n... (truncated)")
    else:
        f.write(summary_str)
    f.write("\n")

print("Analysis complete. Report updated.")
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.signal import correlate
import os
import math

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
# Fix: Removed dtype argument entirely to let pandas infer all column types
    naturally.
# This prevents conversion errors on mixed data columns (e.g., column 30
    containing strings).
df = pd.read_csv(file_path)

# Debug: Inspect columns and first rows to identify correct flow column
print("First few rows:\n", df.head())
print("Column names:\n", df.columns.tolist())

# Note: Column selection and conversion logic is paused until correct column names
     are identified from debug output.
# The following lines are commented out to prevent execution errors in this cycle.
# df['stable_region_mask'] = df['stable_region_mask'].fillna(0).astype('int64')
# cols = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'stable_region_mask', '
    volume_ml', 'System_flow_ml_min']
# df = df[cols].copy()

print("Data loaded successfully. Please review printed column names to update the
    'cols' list in the next cycle.")
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
```

```
from scipy.signal import find_peaks
import os

# Configuration
INPUT_FILE = "/inputs/chromatography_combined.csv"
OUTPUT_FILE = "/workspace/analysis_report.md"
# Removed UV_2_260_mAU from REQUIRED_COLS as it is not used in the simulation
    logic
REQUIRED_COLS = ['run_no', 'UV_1_280_mAU']

# 1. Load Data
try:
    df = pd.read_csv(INPUT_FILE)
except FileNotFoundError:
    raise FileNotFoundError(f"Input file {INPUT_FILE} not found.")

# Check for required columns
missing_cols = [col for col in REQUIRED_COLS if col not in df.columns]
if missing_cols:
    raise ValueError(f"Missing required columns in input data: {missing_cols}")

# 2. Extract Statistics from Step 2
# Calculate  _raw  from the standard deviation of the baseline signal (
    UV_1_280_mAU)
sigma_raw = df['UV_1_280_mAU'].std()

# Retrieve inferred_peak_width from the dataframe
# Feedback: Handle multiple rows by selecting the value corresponding to the
    specific run or raising an error if ambiguous.
# Assuming the dataframe represents a single run or the column contains a single
    unique value for the context.
if 'inferred_peak_width' in df.columns:
    # Check if the column has a single unique value or if we should pick a
        specific row
    # If multiple rows exist, we assume the first row or the mean if they are
        identical.
    # To be safe against multiple different values, we check uniqueness.
    unique_widths = df['inferred_peak_width'].unique()
    if len(unique_widths) > 1:
        raise ValueError("Multiple unique values found for 'inferred_peak_width'.
            Please ensure the input data corresponds to a single run or contains a
            consistent width.")
    inferred_peak_width = int(df['inferred_peak_width'].iloc[0])
else:
    # Fallback: Calculate from data if column is missing (as per feedback fallback
        option)
    peaks, properties = find_peaks(df['UV_1_280_mAU'])
    if len(peaks) > 0 and 'widths' in properties:
        inferred_peak_width = int(np.mean(properties['widths']))
    else:
        inferred_peak_width = 10 # Default fallback

# Ensure inferred_peak_width is valid
if inferred_peak_width <= 0:
    inferred_peak_width = 10

# Initial LOD estimate (3 * sigma)
initial_lod = 3.0 * sigma_raw
# Calculate baseline_mean from actual raw data
```

```
baseline_mean = df['UV_1_280_mAU'].mean()

# 3. Simulation Parameters
num_simulations = 1000
noise_length = 10000
multiplier = 3.0
target_fpr = 0.01

def run_simulation(mult, sigma, width, n_sims, noise_len):
    current_lod = mult * sigma

    # Vectorized batch operation: Generate all noise and peaks at once
    # Shape: (num_simulations, noise_length)
    noise = np.random.normal(loc=baseline_mean, scale=sigma, size=(n_sims,
        noise_len))

    # Generate random peak centers for each simulation
    peak_centers = np.random.randint(width, noise_len - width, size=n_sims)
    # Generate random peak amplitudes
    peak_amps = np.random.uniform(0.5 * current_lod, 2.0 * current_lod, size=
        n_sims)

    # Create x-axis for peak calculation
    x = np.arange(noise_len)

    # Vectorized peak generation
    # Reshape for broadcasting: (n_sims, 1) - (1, noise_len) -> (n_sims, noise_len
        )
    peak_centers_col = peak_centers.reshape(-1, 1)
    peak_amps_col = peak_amps.reshape(-1, 1)

    # Gaussian peak formula: amp * exp(-((x - center)^2) / (2 * (width/2.355)^2))
    std_dev = width / 2.355
    peaks_matrix = peak_amps_col * np.exp(-((x - peak_centers_col) ** 2) / (2 *
        std_dev ** 2))

    signal = noise + peaks_matrix

    # Apply peak detection
    # Note: find_peaks is not batch-vectorized, so we loop over simulations.
    # However, the data generation is fully vectorized, minimizing overhead.
    false_positives = 0

    for i in range(n_sims):
        # Apply peak detection on the i-th signal
        detected_peaks, _ = find_peaks(signal[i], height=current_lod, distance=
            width)

        if len(detected_peaks) > 0:
            # Vectorized distance check against the true peak center for this
                simulation
            distances = np.abs(detected_peaks - peak_centers[i])
            # Count detections that are NOT the true peak (distance > width)
            fp_count = np.sum(distances > width)
            false_positives += fp_count

    fpr = false_positives / n_sims
    return fpr, current_lod
```

```
# 4. Iterative Adjustment
final_multiplier = multiplier
final_lod = initial_lod
final_fpr = 1.0

# Find multiplier such that FPR < 1%
while final_fpr > target_fpr:
    final_multiplier += 0.1
    # Call simulation and capture both FPR and the LOD used in that simulation
    final_fpr, final_lod = run_simulation(final_multiplier, sigma_raw,
        inferred_peak_width, num_simulations, noise_length)

    # Safety break
    if final_multiplier > 10.0:
        break

# 5. Validation Status
# Add Continuity_Flag variable (boolean)
# True indicates a failure state (final_fpr > target_fpr)
Continuity_Flag = final_fpr > target_fpr

validation_status = "Pass" if not Continuity_Flag else "Fail"

# 6. Generate Output
report_content = f"""
### Step 3: False Positive Rate Simulation and Validation
- Final LOD Multiplier: {final_multiplier:.2f}
- Calculated LOD (mAU): {final_lod:.4f}
- Simulated False Positive Rate (%): {final_fpr * 100:.4f}
- Synthetic Peak Width (rows): {inferred_peak_width}
- Continuity Flag (True=Fail): {Continuity_Flag}
- Validation Status: {validation_status}
"""

# Append to analysis_report.md
with open(OUTPUT_FILE, "a") as f:
    f.write(report_content)

print("Step 3 completed. Results appended to analysis_report.md")
```

Listing 3: Code_3